# Limitations of Monolithic Architecture and Quality Attributes Analysis

A Case Study on a BookStore System (Django MVC)

Tran Dinh Que
Software Architecture and Design
Semester II–2026

## Contents

**1 Introduction** **2**

**2 Overview of the BookStore Monolithic System** **2**
2.1 System Structure . . . 2
2.2 Implicit Assumptions . . . 2

**3 Tight Coupling in Monolithic BookStore** **3**
3.1 Definition of Tight Coupling . . . 3
3.2 Tight Coupling in BookStore . . . 3
3.3 Consequences . . . 3

**4 Scalability Limitations** **3**
4.1 Scalability as a System Property . . . 3
4.2 Scaling the Monolithic BookStore . . . 4
4.3 Inefficiency in Practice . . . 4

**5 Deployment Issues** **4**
5.1 Deployment Model . . . 4
5.2 Risks . . . 4

**6 Quality Attributes: Introduction** **5**

**7 Scalability as a Quality Attribute** **5**
7.1 BookStore Analysis . . . 5

**8 Availability as a Quality Attribute** **5**
8.1 Definition . . . 5
8.2 Single Point of Failure . . . 5

**9 Modifiability as a Quality Attribute** **6**
9.1 Definition . . . 6
9.2 Impact of Monolithic Design . . . 6

**10 Summary and Transition** **6**
10.1 Key Lessons . . . 6
10.2 Looking Forward . . . 6

# 1 Introduction

In previous assignments, students designed and implemented a **BookStore system** using the **Monolithic MVC architecture** with Django. The core domain classes include:

- `Customer`
- `Book`
- `Cart`
- `CartItem`

The system is structured using:

- MVC pattern
- Layered architecture (Presentation, Business, Data Access)
- A single relational database

This tutorial aims to:

- Analyze the **limitations of the Monolithic architecture**
- Understand how architectural decisions affect **Quality Attributes**
- Prepare students conceptually for the transition to **Microservices**

**Important note:** Monolithic architecture is not "wrong". Its limitations only become critical when the system grows in size, complexity, and organizational scale.

# 2 Overview of the BookStore Monolithic System

## 2.1 System Structure

The BookStore system follows a typical Django MVC structure:

- **Models**: Customer, Book, Cart, CartItem
- **Views**: Handle HTTP requests and responses
- **Templates**: User interface
- **Database**: Single shared MySQL/PostgreSQL database

All components are deployed as **one application unit**.

## 2.2 Implicit Assumptions

The monolithic design assumes:

- All modules evolve together
- All modules are deployed together
- All modules share the same database schema

These assumptions are acceptable for small systems, but become problematic as the system grows.

# 3 Tight Coupling in Monolithic BookStore

## 3.1 Definition of Tight Coupling

Tight coupling occurs when:

- A module depends directly on internal details of another module
- A change in one module forces changes in others
- Modules cannot be developed or tested independently

## 3.2 Tight Coupling in BookStore

In the BookStore monolithic system:

- `Cart` directly depends on `Customer`
- `CartItem` depends on both `Cart` and `Book`
- Business logic is often embedded directly in Django views

**Example:**

```
CartItem -> Cart -> Customer
CartItem -> Book
```

A change in the `Customer` model (e.g., membership level) may require changes in:

- Cart logic
- Pricing rules
- Checkout workflow

## 3.3 Consequences

- High risk when modifying code
- Difficult refactoring
- Hard to isolate bugs
- Reduced development velocity

### Exercise 1

Identify at least **three places** in your BookStore code where a change in one class forces changes in other classes. Explain why these dependencies exist.

# 4 Scalability Limitations

## 4.1 Scalability as a System Property

Scalability refers to the ability of a system to:

- Handle increased number of users
- Process higher request rates
- Grow without proportional cost increase

### 4.2 Scaling the Monolithic BookStore

The only way to scale the monolithic BookStore is:

- Replicate the entire application
- Place a load balancer in front

### 4.3 Inefficiency in Practice

In real usage:

- Book search and browsing generate most traffic
- Checkout and payment generate much less traffic

However, monolithic scaling requires:

> Scaling **everything**, even if only one module is under heavy load.

#### Exercise 2

Assume:

- 80% of requests are book search
- 10% are cart operations
- 10% are checkout

Explain why scaling the entire BookStore application is inefficient. Which components actually need scaling?

## 5 Deployment Issues

### 5.1 Deployment Model

In a monolithic system:

- Any change requires redeploying the entire application
- All modules are restarted together

### 5.2 Risks

- A bug in Cart may crash the entire BookStore
- Deployment causes downtime or degraded service
- Rollback affects all features

#### Exercise 3

If you fix a minor bug in `CartItem`, list all components that must be retested and redeployed. Explain why this increases operational risk.

# 6 Quality Attributes: Introduction

Chapter 14

Quality Attributes describe **how well** a system performs, rather than **what** it does.

In this tutorial, we focus on:

- Scalability
- Availability
- Modifiability

Architectural choices strongly influence these attributes.

# 7 Scalability as a Quality Attribute

## 7.1 BookStore Analysis

The monolithic BookStore:

- Scales at application level, not domain level
- Cannot scale Book browsing independently
- Suffers from shared database bottlenecks

### Exercise 4

Propose a scenario (e.g., flash sale, holiday traffic) and analyze how the current architecture limits scalability.

# 8 Availability as a Quality Attribute

## 8.1 Definition

Availability measures:

- The proportion of time the system is operational
- Resistance to partial failures

## 8.2 Single Point of Failure

In the monolithic BookStore:

- One application process
- One database

Failure in any core module results in:

**Total service outage**

### Exercise 5

If the Cart module contains a memory leak, explain how this affects:

- Book browsing
- User login
- Checkout

# 9 Modifiability as a Quality Attribute

## 9.1 Definition

Modifiability is the ease with which:

- New features are added
- Existing features are changed
- Bugs are fixed

## 9.2 Impact of Monolithic Design

- High coupling increases change cost
- Developers fear refactoring
- Feature development slows over time

### Exercise 6

Assume the BookStore must support:

> Multiple sellers with independent inventories

Analyze why this change is difficult in the current monolithic design.

# 10 Summary and Transition

## 10.1 Key Lessons

- Monolithic architecture is suitable for small systems
- Tight coupling limits scalability, availability, and modifiability
- Quality Attributes must be considered early in design

## 10.2 Looking Forward

The limitations analyzed in this tutorial motivate the transition to:

- Microservices architecture
- Domain-based decomposition
- Independent deployment and scaling

In the next module, we will redesign the BookStore using a Microservices-based architecture.

Nop tg: 08 Tef

# ASSIGNMENT 04
# Integrated Analysis of Monolithic Limitations

This exercise extends and integrates the previous exercises by analyzing the **BookStore monolithic system as a whole**. Students must reason about **coupling, scalability, and deployment** using their existing Django implementation.

## Context

You have implemented a BookStore system using:

- Django MVC architecture
- Layered design (Presentation, Business, Data Access)
- Core domain classes:
  - `Customer`
  - `Book`
  - `Cart`
  - `CartItem`
- A single shared relational database

The system is deployed as a **single monolithic application**.

Answer the following questions carefully. Your answers must reference:

- Class relationships
- Code dependencies
- Runtime behavior

## Part A — Coupling Analysis

**A1.** Draw a dependency diagram between `Customer`, `Book`, `Cart`, and `CartItem`.

Clearly indicate:

- Direct dependencies
- Indirect dependencies

**A2.** Identify at least **three concrete examples** in your Django code where:

- A change in one class requires changes in others
- Business logic is tightly coupled to data models

**A3.** Consider the following change request:

> "Customers can belong to multiple membership tiers, each with different discount policies that change over time."

Explain:

- Which existing classes must be modified
- Why this change propagates across multiple modules
- How tight coupling increases change cost

## Part B — Scalability Analysis

**B1.** Assume the following workload distribution:

| Operation | Percentage of Requests |
|---|---|
| Book search and browsing | 80% |
| Cart operations | 15% |
| Checkout and payment | 5% |

Explain why scaling the entire monolithic application is inefficient under this workload.

**B2.** Identify which components:

- Consume the most CPU
- Generate the most database queries
- Are actually responsible for performance bottlenecks

**B3.** Suppose traffic suddenly increases by a factor of 10 during a holiday promotion.

Discuss:

- How the monolithic architecture must be scaled
- Which resources are wasted
- Why fine-grained scaling is impossible

## Part C — Deployment and Risk Analysis

**C1.** You fix a small bug in the `CartItem` pricing logic.

List:

- All modules that must be retested
- All components that must be redeployed

Explain why this is required in a monolithic system.

**C2.** During deployment, an unexpected runtime error occurs in the `Cart` module.

Analyze the impact on:

- Book browsing
- User authentication
- Checkout functionality

**C3.** Explain how this deployment model increases:

- Operational risk
- Downtime probability
- Fear of frequent releases

## Part D — Synthesis and Reflection

**D1.** Summarize how:

- Tight coupling
- Coarse-grained scalability
- Monolithic deployment

are structurally related in the BookStore system.

**D2.** Explain why these issues become more severe as:

- The codebase grows
- The number of developers increases
- Business requirements evolve

**D3.** Based on your analysis, briefly explain why architectural evolution (e.g., towards Microservices) becomes necessary rather than optional.

10 pages

**Deliverable:**

- Writing >5 pages PDF
- Using Architecture Diagrams (VP-UML) with RED arrows to show your understand
- Answers must reference the existing BookStore implementation